

Práctica de Laboratorio 6: Telemetría en P4 — Inspección de Ruta de Múltiples Saltos y Medición de Latencia Dentro del Conmutador

Este laboratorio introduce dos técnicas para telemetría de red implementadas completamente en el plano de datos de P4. Ambos ejercicios van más allá del reenvío: los propios paquetes transportan datos de medición que los conmutadores escriben a velocidad de línea, sin participación del plano de control durante la medición. Se aplica el mismo flujo de trabajo de compilación y despliegue de los laboratorios anteriores: compilar con `p4c-bm2-ss`, iniciar la topología de Mininet con `topo.py` e instalar las reglas de reenvío usando `simple_switch_CLI`.

El ejercicio guiado (MRI) implementa una variante simplificada de Telemetría en Banda (INT). Cada conmutador atravesado por un paquete añade su propio identificador y la profundidad actual de la cola a una pila de cabeceras en crecimiento transportada dentro del paquete IP como una opción IPv4. El receptor puede leer la secuencia completa de saltos y su estado de congestión por salto directamente del paquete que llega, sin consultar ningún sistema de monitoreo externo.

La actividad del estudiante (MySec) implementa un protocolo personalizado para la medición de latencia dentro del conmutador. Un paquete sigue una ruta de rebote — `h1` a `s1` a `s2` de regreso a `s1` y finalmente de regreso a `h1` — mientras que los conmutadores registran marcas de tiempo de procesamiento en campos de cabecera dedicados. El emisor recupera el paquete y lee los valores de latencia sin necesidad de coordinación fuera de banda. Una diferencia clave con MRI es que MySec usa un número de protocolo IP personalizado (169) para activar la lógica del plano de datos, mientras que MRI se basa en un tipo de opción IPv4.

Ejercicio 1: MRI — Inspección de Ruta de Múltiples Saltos (Guiado)

MRI es una implementación simplificada de Telemetría en Banda (INT). INT es un marco definido por el consorcio P4.org que permite a los dispositivos de red incrustar datos de monitoreo directamente en paquetes del plano de datos a velocidad de línea, sin generar mensajes de control separados ni copiar

paquetes a un recolector. En su forma completa, INT transporta información por salto como ID del conmutador, profundidad de cola, marcas de tiempo de ingreso y egreso, y utilización del enlace. MRI implementa la idea central: una pila de cabeceras en crecimiento en la que cada conmutador inserta su ID y la profundidad actual de la cola de salida.

El mecanismo se basa en opciones IPv4. Cada paquete que transporta datos MRI tiene su campo `ihl` configurado por encima de 5 (el mínimo para una cabecera IPv4 simple). El tipo de opción 31 señala la presencia de una cabecera MRI, que a su vez contiene un campo `count` y una pila de registros de conmutador (`swid`, `qdepth`). Cada conmutador que procesa el paquete en egreso llama a `push_front(1)` para aumentar la pila en una entrada, escribe su propio identificador y profundidad de cola, e incrementa `ihl`, `totalLen` y `optionLength` en consecuencia. El receptor lee la pila completa, que lista los conmutadores en orden inverso a la ruta (último salto primero), obteniendo tanto la ruta que recorrió el paquete como el estado de congestión en cada nodo.

La topología consta de tres conmutadores P4 y cinco hosts. Dos hosts están conectados a `s1` (subred 10.0.1.0/24), dos a `s2` (subred 10.0.2.0/24) y uno a `s3` (subred 10.0.3.0/24). El enlace entre `s1` y `s2` tiene una tasa limitada a 0.5 Mbps, actuando como un cuello de botella de congestión que puede saturarse con tráfico de fondo para producir valores observables de profundidad de cola en los registros MRI. El tercer conmutador proporciona una ruta alternativa, permitiendo al receptor distinguir diferentes rutas a partir de los identificadores de conmutador transportados en la pila.

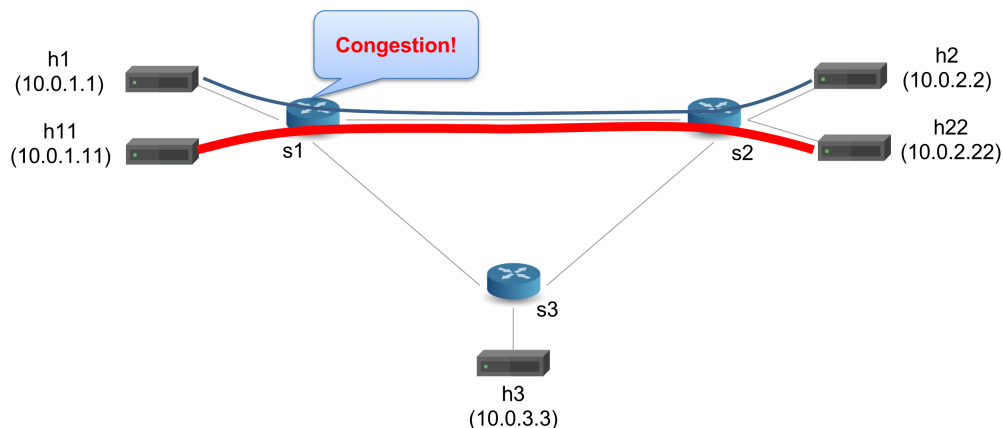


Figura 1: Topología MRI: tres conmutadores P4, cinco hosts y un cuello de botella de 0.5 Mbps entre s1 y s2.

Las asignaciones de hosts y puertos son:

- s1 (puerto Thrift 9090): h1 en 10.0.1.1/24 en el puerto 1, h11 en 10.0.1.11/24 en el puerto 2, s2 en el puerto 3 (cuello de botella), s3 en el puerto 4.
- s2 (puerto Thrift 9091): h2 en 10.0.2.2/24 en el puerto 1, h22 en 10.0.2.22/24 en el puerto 2, s1 en el puerto 3 (cuello de botella), s3 en el puerto 4.
- s3 (puerto Thrift 9092): h3 en 10.0.3.3/24 en el puerto 1, s1 en el puerto 2, s2 en el puerto 3.

El ejercicio utiliza reenvío L3. Cada host está configurado con una ruta por defecto a una puerta de enlace virtual, y el conmutador reescribe la MAC de destino Ethernet en cada salto. topo.py preconfigura entradas ARP estáticas en cada host para que el tráfico entre subredes se resuelva correctamente sin inundación ARP. Herramientas estándar como ping producen paquetes IPv4 estándar sin opciones MRI; los scripts proporcionados send.py y receive.py son necesarios para crear y analizar las cabeceras personalizadas.

Paso 1: Haga ejecutables los scripts auxiliares y compile el programa P4. Desde la carpeta mri en el repositorio del laboratorio:

```
chmod +x send.py receive.py
```

```
mkdir -p p4src/build
```

```
p4c-bm2-ss --p4v 16 -o p4src/build/bmv2.json p4src/mri.p4
```

El compilador traduce mri.p4 a bmv2.json. Puede aparecer una advertencia de obsolescencia sobre el formato de salida que se puede ignorar. A diferencia de los laboratorios anteriores, este ejercicio no requiere la bandera `--p4runtime-files`, ya que el plano de control usa `simple_switch_CLI` directamente.

Paso 2: Iniciar la topología de Mininet:

```
sudo python3 mininet/topo.py
```

El script de topología inicia tres instancias de BMv2 (s1 en el puerto Thrift 9090, s2 en el puerto 9091, s3 en el puerto 9092), crea cinco hosts con ARP estático y rutas de puerta de enlace, y configura el enlace de cuello de botella de 0.5 Mbps entre s1 y s2. Deje esta terminal abierta.

Paso 3: En una segunda terminal, instale las reglas de reenvío en los tres conmutadores. Cada conmutador requiere dos tipos de entradas: una acción predeterminada para la tabla `swtrace` de egreso y entradas LPM para la tabla `ipv4_lpm` de ingreso.

```
simple_switch_CLI --thrift-port 9090 <s1-commands.txt
```

```
simple_switch_CLI --thrift-port 9091 <s2-commands.txt
```

```
simple_switch_CLI --thrift-port 9092 <s3-commands.txt
```

Este ejercicio introduce un nuevo comando CLI: `table_set_default`. A diferencia de `table_add`, que instala una entrada de coincidencia, `table_set_default` establece la acción que se ejecuta cuando no existe una entrada que coincida — o, en este caso, cuando la tabla no tiene ninguna clave. La tabla de egreso `swtrace` no tiene campos de coincidencia; cada paquete que llega al pipeline de egreso con una cabecera MRI válida la activa incondicionalmente. El comando asigna un identificador de conmutador único a cada instancia de BMv2:

```
table_set_default MyEgress.swtrace MyEgress.add_swtrace 1
```

Cada archivo de comandos también contiene entradas LPM para la tabla `ipv4_lpm`. s1 y s2 reciben cada uno cuatro entradas: dos entradas /32 para

sus hosts directamente conectados, y dos entradas /24 para las subredes remotas reenviadas a través de los enlaces entre conmutadores. s3 recibe tres entradas: una entrada /32 para h3 y dos entradas /24 para las subredes remotas accesibles a través de s1 y s2. Las entradas LPM siguen la sintaxis introducida en los laboratorios anteriores:

```
table_add MyIngress.ipv4_lpm MyIngress.ipv4_forward  
<ip>/<prefijo>=><mac-siguiente-salto><puerto>
```

Paso 4: Verificar la conectividad desde la CLI de Mininet:

```
mininet>pingall
```

El resultado esperado es 20 % de pérdida (16 de 20 pares tienen éxito). Los cuatro pares que fallan son h1–h11 (ambos en s1 en la subred 10.0.1.0/24) y h2–h22 (ambos en s2 en la subred 10.0.2.0/24), en ambas direcciones. Debido a que estos pares de hosts comparten la misma subred /24, la pila de red Linux en cada host intenta resolver la IP destino mediante ARP directamente, sin enrutar a través de la puerta de enlace. El conmutador P4 no responde a las solicitudes ARP, por lo que la resolución falla y ping no puede continuar. Este comportamiento es correcto y esperado; todos los pares entre conmutadores deben tener éxito antes de continuar.

Paso 5: Abra terminales de host adicionales y ejecute la prueba MRI. Desde la CLI de Mininet:

```
mininet>xterm h1 h2 h11 h22
```

En la terminal de h22, inicie un servidor UDP para absorber el tráfico de fondo:

```
h22# iperf -s -u
```

En la terminal de h2, inicie el receptor MRI. Este script usa Scapy con un disector personalizado IOption_MRI que analiza el tipo de opción MRI y la pila de cabeceras SwitchTrace:

```
h2# ./receive.py
```

En la terminal de h11, inicie un flujo de fondo saturante hacia h22 para llenar la cola del cuello de botella:

```
h11# iperf -c 10.0.2.22 -t 15 -u
```

Inmediatamente después, en la terminal de h1, envíe paquetes etiquetados con MRI hacia h2:

```
h1# ./send.py 10.0.2.2 "P4 is cool"30
```

El script send.py construye paquetes UDP que incluyen una opción IPv4 de tipo 31 con count=0 y una lista swtrace vacía. Esto señala a cada conmutador en la ruta que el paquete transporta datos MRI. Cada conmutador añade su propio registro durante el procesamiento de egreso. El receptor en h2 imprime el paquete decodificado para cada trama que llega usando la función pkt.show2() de Scapy. La sección relevante de cada impresión es el campo options, que muestra la estructura MRI:

```
got a packet
###[ MRI ]###
  option      = 31
  length      = 20
  count       = 2
  \swtraces   \
    |###[ SwitchTrace ]###
    |  swid      = 2
    |  qdepth    = 1
    |###[ SwitchTrace ]###
    |  swid      = 1
    |  qdepth    = 0
```

El campo count muestra cuántos conmutadores insertaron un registro. La pila está ordenada del más reciente al primero: la primera entrada (swid=2) fue insertada por s2 (el último conmutador antes de h2), y la segunda entrada (swid=1) fue insertada por s1. Mientras el flujo de fondo desde h11 satura el cuello de botella s1–s2, el valor de qdepth para swid=1 debería elevarse por encima de cero, reflejando la cola de salida creciente en s1. Después de que el flujo iperf termina y la cola se vacía, los valores de qdepth deberían volver a cero o uno.

Paso 6: Verificar la visibilidad de la ruta en una ruta alternativa. Abra una terminal de host para h3 y repita la prueba en la ruta que atraviesa s3 en lugar de s2:

```
mininet>xterm h3
```

```
h3# ./receive.py
```

```
h1# ./send.py 10.0.3.3 "P4 is cool"5
```

Esta vez la ruta es $h1 \rightarrow s1 \rightarrow s3 \rightarrow h3$, y la pila swtrace en h3 mostrará swid=3 (el más reciente) y swid=1. Este contraste es la observación central de la Inspección de Ruta de Múltiples Saltos: la pila de cabeceras revela qué conmutadores fueron realmente atravesados, sin ninguna infraestructura de monitoreo externa. Ningún paquete en esta topología atraviesa los tres conmutadores simultáneamente; la diferencia de ruta está codificada directamente en los identificadores de conmutador transportados dentro del paquete.

Descripción General del Pipeline P4

El programa P4 (p4src/mri.p4) implementa reenvío LPM IPv4 en el pipeline de ingreso y manipulación de la pila MRI en el pipeline de egreso. El programa define las cabeceras específicas de MRI junto con las cabeceras estándar Ethernet e IPv4:

```
header ipv4_option_t {  
    bit<1>  copyFlag;  
    bit<2>  optClass;  
    bit<5>  option;  
    bit<8>  optionLength;  
}
```

```
header mri_t {  
    bit<16> count;  
}
```

```
header switch_t {
```

```

switchID_t swid;
qdepth_t qdepth;
}

struct headers {
    // ...
    switch_t[MAX_HOPS] swtraces; } }

```

El analizador usa una cadena de dos estados para consumir los datos MRI. El estado `parse_mri` extrae la cabecera MRI base, almacena el count en los metadatos del analizador y transita a `parse_swtrace` si hay registros presentes. El estado `parse_swtrace` es recursivo: extrae una cabecera `switch_t` por llamada, decrementa el contador restante y vuelve a sí mismo hasta que el contador llega a cero:

```

state parse_mri {
    packet.extract(hdr.mri);
    meta.parser_metadata.remaining = hdr.mri.count;
    transition select(meta.parser_metadata.remaining) {
        0 : accept;
        default: parse_swtrace;
    }
}

state parse_swtrace {
    packet.extract(hdr.swtraces.next);
    meta.parser_metadata.remaining =
        meta.parser_metadata.remaining - 1;
    transition select(meta.parser_metadata.remaining) {
        0 : accept;
        default: parse_swtrace; // recursivo
    }
}

```

El control de egreso define la acción `add_swtrace`, que antepone una nueva

entrada a la pila y ajusta los campos de longitud de la cabecera IPv4 para contabilizar los ocho bytes adicionales (4 para swid, 4 para qdepth):

```
action add_swtrace(switchID_t swid) {
    hdr.mri.count = hdr.mri.count + 1;
    hdr.swtraces.push_front(1); // aumenta la pila en uno
    hdr.swtraces[0].setValid();
    hdr.swtraces[0].swid = swid;
    hdr.swtraces[0].qdepth =
        (qdepth_t) standardmetadata.deq_qdepth;
    hdr.ipv4.ihl = hdr.ipv4.ihl + 2;
    hdr.ipv4_option.optionLength =
        hdr.ipv4_option.optionLength + 8;
    hdr.ipv4.totalLen = hdr.ipv4.totalLen + 8;
}
```

La tabla swtrace no tiene clave de coincidencia; ejecuta incondicionalmente add_swtrace a través de su acción predeterminada cuando hdr.mri es válido. Esta es la razón por la que el plano de control instala el identificador de conmutador usando table_set_default en lugar de table_add. Debido a que ihl y totalLen cambian en cada paquete, MyComputeChecksum recalcula el checksum IPv4 con los valores actualizados de los campos.

Actividad 1: MySec — Medición de Latencia Dentro del Conmutador

MySec implementa un protocolo personalizado de medición de latencia dentro del conmutador. Un paquete construido por el emisor lleva una cabecera de 8 campos (mysec_t) y recorre una ruta de rebote predefinida: $h1 \rightarrow s1 \rightarrow s2 \rightarrow s1 \rightarrow h1$. Mientras está en tránsito, s1 registra marcas de tiempo de egreso e ingreso directamente en la cabecera del paquete en cada uno de sus dos pasos; s2 actúa como un reflector puro y no modifica la cabecera. Cuando el paquete regresa a h1, el emisor lee los valores de latencia de la trama que llega sin necesidad de comunicación fuera de banda.

El protocolo se activa mediante el número de protocolo IP 169, un valor no estándar sin asignación real. El plano de datos usa este valor para identificar los paquetes MySec en el analizador y el pipeline de ingreso, enrutándo-

los a través de la tabla TS_table dedicada en lugar de la tabla de reenvío L2 estándar; solo los paquetes que llevan una cabecera mysec.t válida como resultado de esta clasificación pueden posteriormente activar la lógica de marcas de tiempo en el pipeline de egreso. Las herramientas estándar que generan tráfico IPv4 (ping, iperf, hping3) usan números de protocolo diferentes y se reenvían normalmente; solo el script proporcionado send_mysec.py genera la estructura de trama correcta. La ruta de rebote es impuesta por el plano de control: s2 está configurado para redirigir los paquetes de vuelta hacia s1 en lugar de entregarlos a h2, independientemente del estado del paquete en el momento de la llegada.

La topología consta de dos conmutadores P4 y dos hosts dispuestos en una cadena lineal:

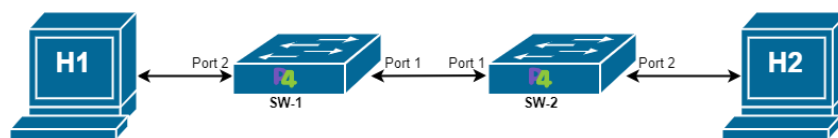


Figura 2: Topología MySec: h1 y h2 conectados a través de s1 y s2. Un paquete MySec entra a s1 desde h1, cruza a s2, es redirigido de vuelta a s1 y finalmente regresa a h1.

- s1 (puerto Thrift 9090): h1 en 10.0.0.1/24 en el puerto 1, s2 en el puerto 2.
- s2 (puerto Thrift 9091): s1 en el puerto 1, h2 en 10.0.0.2/24 en el puerto 2.

Todos los enlaces están configurados con ancho de banda de 5 Mbps, retardo de 5 ms y pérdida de paquetes del 1 %. Ambos hosts están en la misma subred /24, y topo.py proporciona entradas ARP estáticas. El tráfico estándar (no MySec) se reenvía mediante l2_exact.table usando la MAC destino como única clave de coincidencia. El tráfico MySec (proto = 169) omite por completo l2_exact.table y es manejado por una tabla separada de dos claves (TS_table) que coincide tanto con la MAC destino como con el puerto de ingreso.

Cabecera MySec

La cabecera `mysec_t` ocupa 304 bits (38 bytes) y contiene ocho campos:

```
header mysec_t {  
    bit<4>  ingress_port;  
    bit<4>  egress_port;  
    bit<48> process_time_sw1;  
    bit<48> process_time_sw2;  
    bit<48> egress_time_sw1;  
    bit<48> ingress_back_time_sw1;  
    bit<48> total;  
    bit<48> th;  
}
```

Los campos `ingress_port` y `egress_port` son marcadores de máquina de estados, no números de puerto físicos. Permiten que el pipeline de egreso de `s1` distinga si el paso de procesamiento actual es el trayecto de ida (`h1` hacia `s2`) o el trayecto de retorno (`s2` hacia `h1`). Los seis campos de 48 bits almacenan valores relacionados con temporización, pero su semántica difiere: `egress_time_sw1` e `ingress_back_time_sw1` son marcas de tiempo absolutas registradas a partir de `standard_metadata.egress_global_timestamp` y `standard_metadata.ingress_global_timestamp` respectivamente; `process_time_sw1`, `process_time_sw2` y `total` son duraciones de tiempo de procesamiento derivadas de diferencias o sumas de marcas de tiempo, expresadas en nanosegundos; y `th` es una bandera de 0/1 que indica si el total acumulado excede un límite configurado. Todos los valores se escriben en el pipeline de egreso, que es el primer punto en el modelo `v1model` donde ambas marcas de tiempo globales están disponibles simultáneamente para un tránsito de paquete dado. Nótese que, a pesar de la convención de nombres, `process_time_sw2` es calculado por `s1` en el trayecto de ida; el subíndice refleja la dirección del viaje, no el conmutador físico que realiza el cálculo.

Ciclo de Vida del Paquete

La Figura 3 ilustra el modelo temporal conceptual de la ruta de rebote.

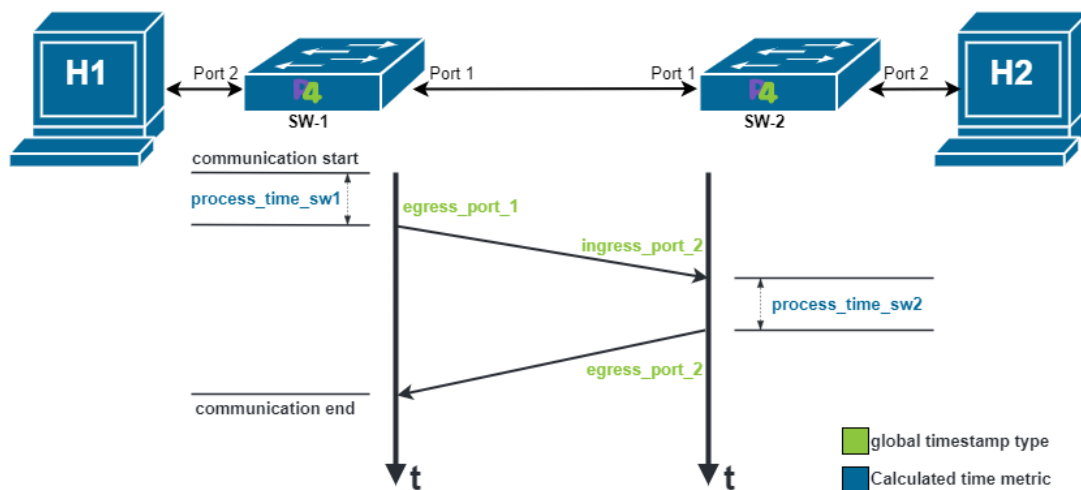


Figura 3: Diagrama de tiempos de MySec: ruta de rebote $h1 \rightarrow SW-1 \rightarrow SW-2 \rightarrow SW-1 \rightarrow h1$. SW-1 escribe marcas de tiempo en ambos pasos. SW-2 actúa como un reflector puro y no calcula ninguna marca de tiempo.

El paquete inicial enviado por `send_mysec.py` lleva `ingress_port = 1`, `egres_port = 2` y ceros en todos los campos de marca de tiempo. El paquete entra a s1 en el trayecto de ida (llegando por el puerto 1) y es dirigido al puerto 2 (hacia s2) por la tabla `TS_table` de ingreso. En el pipeline de egreso, s1 detecta la condición de trayecto de ida y calcula la latencia de egreso para ese paso, escribiéndola en `process_time_sw2`. También actualiza los campos de la máquina de estados y reescribe la MAC destino Ethernet para preparar el paquete para la ruta de retorno.

El paquete llega a s2. La implementación P4 en s2 no escribe marcas de tiempo: no evalúa condiciones de marca de tiempo y no escribe nada en la cabecera `mysec`. Simplemente aplica `TS_table` y redirige el paquete de vuelta hacia s1 por el puerto 1. El paquete nunca se entrega a h2.

El paquete reingresa a s1 en el trayecto de retorno (llegando por el puerto 2). En el pipeline de egreso, s1 detecta la condición de trayecto de retorno y calcula la latencia de egreso para este paso, escribiéndola en `process_time_sw1`. También registra la marca de tiempo de egreso absoluta y las tres métricas adicionales de la Parte 2 del TODO. El paquete sale hacia h1 por el puerto 1, y `send_mysec.py` captura la trama de retorno, imprimiendo todos los campos de marca de tiempo completados.

Tablas de Reenvío

El bloque apply de ingreso selecciona la tabla de reenvío según el protocolo IP:

```
if (hdr.ipv4.protocol != PROTOCOL_MYSEC) {  
    l2_exact_table.apply();  
} else {  
    TS_table.apply(); // dos claves: dst_MAC + ingress_port  
}
```

La tabla TS_table requiere dos entradas en s1 y dos entradas en s2. En s1, la primera entrada reenvía paquetes MySec destinados a h2_MAC que llegan por el puerto 1 hacia el puerto 2 (hacia s2, trayecto de ida); la segunda entrada reenvía paquetes MySec destinados a h1_MAC que llegan por el puerto 2 hacia el puerto 1 (hacia h1, trayecto de retorno). En s2, ambas entradas redirigen al puerto 1 (de vuelta hacia s1), porque la ruta de rebote nunca entrega un paquete MySec directamente a h2. La coincidencia de dos claves (dst_MAC, ingress_port) es necesaria porque s1 procesa el mismo paquete dos veces: sin la clave ingress_port, el conmutador no puede determinar si un paquete MySec dado está en el trayecto de salida o de retorno.

Secciones TODO para Completar

Abra p4src/main.p4 de la carpeta mysec en el repositorio del laboratorio. El programa contiene dos regiones TODO en el control EgressPipeImpl, ambas dentro del bloque que procesa paquetes MySec.

El primer TODO (**Exercise 3 TO-DO**) requiere completar las condiciones basadas en puerto que distinguen los dos pasos de s1. El control de egreso usa una variable local para representar el número de puerto desde el cual llegó el paquete en el trayecto de retorno. El comentario indica que asigne a esta variable el número de puerto correcto para que la rama condicional para el trayecto de retorno se active en el segundo paso a través de s1. Una segunda rama, estructurada como else-if, maneja el trayecto de ida y también debe referenciar el puerto correcto. Consulte el diagrama de tiempos en el repositorio para identificar qué puerto debe verificar cada condición. La misma región

TODO también requiere completar la reescritura de la dirección MAC destino Ethernet para que, después del procesamiento del trayecto de ida, el paquete sea dirigido de vuelta hacia h1 y no hacia h2.

El segundo TODO (**Exercise 3 TO-DO, Part 2 — Additional Metrics**) está anidado dentro de la rama del trayecto de retorno. Hay tres líneas comentadas, cada una con un marcador de posición que debe ser reemplazado por una expresión:

- `ingress_back_time_sw1`: registre la marca de tiempo de ingreso absoluta de s1 en la ruta de retorno. Use el campo de marca de tiempo `standard_metadata` apropiado disponible en el pipeline de egreso.
- `total`: calcule el tiempo de procesamiento combinado en ambas direcciones. Use los dos campos `process_time` ya escritos en pasos anteriores.
- `th`: evalúe si el total excede un umbral de su elección y escriba 1 o 0 en este campo. Esto implementa una decisión del plano de datos basada en telemetría acumulada sin intervención del controlador.

No modifique el analizador, el control de ingreso, las tablas de reenvío, el bloque de checksum ni el desanalizador. El esqueleto ya proporciona la estructura completa; solo es necesario completar las expresiones dentro de las regiones TODO.

Compilar y Desplegar

Una vez completada la implementación, compile el programa desde la carpeta `mysec`:

```
mkdir -p p4src/build
p4c-bm2-ss --p4v 16 -o p4src/build/bmv2.json \
--p4runtime-files p4src/build/p4info.txt p4src/main.p4
```

Inicie la topología de Mininet:

```
sudo python3 mininet/topo.py
```

En una segunda terminal, instale las reglas de reenvío en ambos conmutadores:

```
simple_switch_CLI --thrift-port 9090 <s1-commands.txt
```

```
simple_switch_CLI --thrift-port 9091 <s2-commands.txt
```

Cada archivo de comandos contiene dos entradas para `l2_exact_table` (reenvío L2 estándar para tráfico ping) y dos entradas para `TS_table` (reenvío consistente del contexto para tráfico MySec). Verifique que todas las entradas sean aceptadas sin errores antes de continuar.

Verificar Conectividad y Ejecutar la Prueba MySec

Confirme que la conectividad estándar funciona antes de probar MySec:

```
mininet>pingall
```

El resultado esperado es 0 % de pérdida: h1 y h2 están en la misma subred /24 y comparten entradas ARP estáticas configuradas por `topo.py`, por lo que el tráfico ICMP estándar puede alcanzar ambos hosts sin enrutamiento por puerta de enlace. Si algún par falla, revise las entradas de `l2_exact_table` antes de continuar.

Envíe un paquete MySec desde h1 y observe el resultado:

```
mininet>h1 python3 mininet/send_mysec.py
```

El script imprime el paquete saliente (con todos los campos de marca de tiempo en cero) y el paquete de retorno (con las marcas de tiempo completadas por los conmutadores). Una ejecución exitosa produce valores distintos de cero en `process_time_sw1` y `process_time_sw2` y devuelve un bloque de resultados con formato:

```
=====
MySec | In-switch latency results
=====

process_time_sw1      : XXXX ns
process_time_sw2      : XXXX ns
egress_time_sw1       : XXXXXXXXXXXX
```



```
ingress_back_time_sw1 : XXXXXXXXXXXX
total                  : XXXX
th                     : 0 or 1
```

=====

Los valores típicos para `process_time_sw1` y `process_time_sw2` en un conmutador software BMv2 ejecutándose en una máquina virtual varían de 500 a 50,000 nanosegundos. Ambos valores deben ser significativamente menores que 5,000,000 ns (el retardo de enlace configurado) para que la medición refleje el tiempo de procesamiento del conmutador en lugar del retardo de propagación del enlace.

Si el script informa que no se recibió respuesta, verifique que `s2-commands.txt` contenga una entrada para `h1_MAC` en `TS_table`. Esta entrada es necesaria porque `s1` modifica la MAC destino Ethernet antes de que el paquete llegue a `s2`, y `s2` debe coincidir con la dirección actualizada para redirigir correctamente el paquete de vuelta hacia `s1`.

Preguntas de Discusión

1. En MRI, el conmutador modifica `ihl`, `totalLen` y `optionLength` en cada paquete que pasa por el egreso. En MySec, ningún campo de la cabecera IPv4 es modificado por el pipeline de reenvío o medición. Sin embargo, en ambos casos los programas P4 incluyen un bloque de cálculo de checksum. Explique qué programa requiere un recálculo activo del checksum y por qué el otro puede dejar ese bloque vacío o comentado.
2. MRI lee `standard_metadata.deq_qdepth` en el pipeline de egreso. MySec lee `standard_metadata.egress_global_timestamp` y `standard_metadata.ingress_global_timestamp`, también en egreso. Explique por qué ambas mediciones deben tomarse en egreso en lugar de ingreso, e identifique qué información no estaría disponible o sería incorrecta si se intentara la misma lógica en el pipeline de ingreso.
3. En el ejercicio MRI, un paquete que viaja de `h1` a `h2` produce un `sw-trace` con `count=2` y registros para `swid=1` y `swid=2`. Un paquete de

h1 a h3 también produce count=2, pero con registros para swid=1 y swid=3. Ningún paquete en esta topología produce count=3. Explique qué implica el término Inspección de Ruta de Múltiples Saltos sobre la observabilidad, y describa un cambio de topología o política de enrutamiento que resultaría en count=3 para algunos flujos.

4. El protocolo MySec usa el número de protocolo IP 169, que no tiene asignación IANA. MRI usa el tipo de opción IPv4 31. Ambos mecanismos son incompatibles con enrutadores y cortafuegos estándar en redes de producción. Describa un mecanismo de nivel productivo estandarizado por la comunidad P4.org que aborde los mismos objetivos de telemetría manteniendo la compatibilidad con la infraestructura IP existente.
5. Tanto MRI como MySec están limitados por el tamaño de un solo paquete: MRI está limitado por MAX_HOPS (9 registros), y MySec reporta solo dos muestras de latencia por viaje redondo. Si cualquiera de estos mecanismos se extendiera a una topología con decenas de conmutadores, ¿qué restricciones de escalabilidad surgirían con respecto al crecimiento del tamaño del paquete, la complejidad de los estados del analizador y la corrección de las mediciones recopiladas?